

The Token Survival Kit

Your low-limit playbook — stop burning tokens, dodge the wall & train Claude's memory. Real commands, repos & skills — field-tested, key repos linked below.

① NATIVE COMMANDS — FREE, BUILT IN

CMD /compact

Summarize the whole chat into a compact state & keep going — frees the context window.

CMD /clear

Wipe context when you switch to unrelated work. Stops old tokens riding along.

CMD Subagents

Delegate noisy jobs — verbose output stays in the subagent, only a summary returns.

CMD /usage

Check live token spend mid-session before you blow the budget.

CMD /effort low

Skip deep reasoning on easy tasks. Lower effort = fewer thinking tokens.

CMD Plan mode

Approve the plan once (Shift+Tab) — no wasted billable re-do.

CMD /context

Visualize what's filling the context window + where to trim it.

② GITHUB REPOS — INSTALL & RUN

REPO ccusage

See exactly what you burned, in \$, by day/session/project. Local logs, nothing uploaded.

```
npx ccusage@latest daily
```

github.com/ryoppiippi/ccusage

REPO Usage-Monitor

Real-time monitor that predicts & warns you BEFORE you hit the rate limit.

```
pip install claude-monitor
```

github.com/Maciek-roboblog/Claude-Code-Usage-Monitor

REPO repomix --compress

Pack a repo into one file. --compress (Tree-sitter) → ~70% fewer tokens.

```
npx repomix --compress
```

github.com/yamadashy/repomix

REPO repomix --token-budget

Hard cap — fails (non-zero exit) if packed output exceeds N tokens. A guard for agent runs.

```
npx repomix --token-budget 50000
```

③ MEMORY & SKILLS — TRAIN IT ONCE

MEMORY CLAUDE.md

Standing rules auto-loaded each session. Keep tight — every line is paid every session.

```
/init → edit CLAUDE.md
```

MEMORY File-memory

Thin manifest + on-demand notes so Claude recalls projects across sessions.

SKILL Skills > re-prompt

Repeated workflow → one Skill. Write it once, never re-paste.

SKILL Caveman mode

Terse in/out, drops filler. ~70% lighter I/O, full accuracy.

SKILL /compress memory

Shrink CLAUDE.md / memory files into compact form to cut input tokens.

CONFIG Start on Sonnet

Default Sonnet; only call Opus for genuinely deep refactors.

CONFIG MAX_THINKING_TOKENS

Cap the reasoning budget so thinking can't run away.

```
MAX_THINKING_TOKENS=8000
```

CONFIG Prompt caching

Reuse a cached prefix — pay once, then cheap cache hits.

CONFIG Read sections

Read only the lines you need; don't re-read whole files every turn.

Pro stack: Run **ccusage** to see where tokens go → trim **CLAUDE.md** & watch **/context** → push noisy work to **subagents** → **/compact** at every milestone, **/clear** between tasks. That combo alone routinely halves spend before you touch model settings.